

An Efficient Ranking Algorithm of t -ary Trees in Gray-code Order*

Ro-Yu Wu¹ Jou-Ming Chang^{2,†} An-Hang Chen² Chun-Liang Liu³

¹ Department of Industrial Management,
Lunghwa University of Science and Technology, Taoyuan, Taiwan, ROC

² Institute of Information and Decision Sciences,
National Taipei College of Business, Taipei, Taiwan, ROC

³ Department of Humanity & Social Science,
National Taiwan Normal University, Taipei, Taiwan, ROC

Abstract

A t -ary tree is a rooted tree such that every internal node has exactly t disjoint subtrees. Recently, a concise representation called RD-sequences was introduced to represent t -ary trees and their generalization called non-regular trees. In particular, a loopless algorithm has been proposed by Wu et al. (2010) for generating non-regular trees (and thus of t -ary trees) encoded by RD-sequences in a Gray-code order. In this paper, based on such a Gray-code order, we present efficient ranking algorithm of t -ary trees with n internal nodes. The time complexity and space requirement in the algorithm are $O(\max\{n^2, tn\})$ and $O(tn)$, respectively. As a by-product, we have an improvement on ranking t -ary trees encoded by z -sequences in a Gray-code order.

Keywords: ranking algorithms; lexicographic order; Gray-code order; t -ary trees;

1. Introduction

Exhaustively generating a class of combinatorial objects has attracted a lot of research due to the importance for many applications, e.g., searching for the desired object with a certain feature among all candidates, looking for a counterexample to some conjecture, or analyzing the average perfor-

mance of an algorithm over all possible inputs. As usual, combinatorial objects are encoded as integer sequences so that these sequences can be generated in a particular order such as the lexicographic order [3, 4, 12, 20] or a Gray-code order [2, 5–10, 15, 17, 19]. For efficiency of generation, algorithms to generate objects in the lexicographic order are demanded to run in a constant amortized time. By contrast, algorithms to generate objects in a Gray-code are demanded to run in a constant time for each generation, i.e., a *loopless algorithms*. Accordingly, a loopless algorithm is implemented non-recursively by using, after the initialization of the first object, only assignment statements and “if-then-else” statements. The reader is referred to [13] for an excellent survey of generating combinatorial objects in Gray-code orders.

Given a specific order on the class of combinatorial objects, a *ranking algorithm* is a function that determines the rank of a given object in the exhaustive generated list, and an *unranking algorithm* is one that generates the object (or its representation) corresponding to a given rank. Trees are one of the most important and fundamental combinatorial objects in computer science. For $t \geq 2$, a *t -ary tree* is a rooted tree such that every internal node has exactly t disjoint subtrees. Many algorithms have been developed for generating t -ary trees (as well as binary trees) with n internal nodes. Moreover, many ranking algorithms [1, 2, 4, 11, 12, 14, 16, 18, 21] and unranking algorithms [1, 2, 4, 12, 16, 18] for diverse representations of t -ary trees with n internal nodes have been proposed. Among all representations of t -ary trees, a well-formed representation

*This research was partially supported by National Science Council of Taiwan under the Grants NSC100-2221-E-262-019 and NSC100-2221-E-141-002.

[†]Corresponding author. (Email:spade@mail.ntcb.edu.tw)

called *z-sequences* was first introduced by Zaks [20]. Roelants van Baronaigien [10] and Xiang et al. [19] independently presented a loopless algorithm for generating t -ary trees encoded by z -sequences in a Gray-code order. Based on such a Gray-code order, Ahmadi-Adl et al. [2] recently presented $O(tn^2)$ -time ranking and unranking algorithms.

To the best of our knowledge, it seems that most of ranking and unranking algorithms for t -ary trees with n internal nodes require $O(tn^2)$ time and space. Recently, Wu et al. [18] introduced a new type of integer sequences called RD-sequences to represent t -ary trees and showed that, using such a representation in lexicographic order, both ranking and unranking problems for t -ary trees with n internal nodes can be solved in $O(tn)$ time. Moreover, they showed that there is a complementary relationship between RD-sequences and z -sequences (see Theorem 1). Indeed, RD-sequences have been used to represent a wider class of trees called *non-regular trees* [16,17], i.e., ordered trees whose internal nodes have a pre-specified degree sequence called *branching sequence* in preorder. With the help of a coding tree structure, Wu et al. [17] presented a loopless algorithm to generate RD-sequences of non-regular trees (and thus of t -ary trees) in a Gray-code order using $4n+O(1)$ space. Later on, Wu et al. [16] dealt with the ranking and unranking problems for non-regular trees encoded by RD-sequences in lexicographic order and showed that, given a prescribed branching sequence $(s_1, s_2, \dots, s_n)$, both ranking and unranking problems can be solved in $O(nS)$ time, where $S = \sum_{i=1}^{n-1} (s_i - 1)$.

In this paper, based on the Gray-code order of t -ary trees encoded by RD-sequences in [17], we present an efficient ranking algorithm. The time complexity and space requirement in our algorithm are $O(\max\{n^2, tn\})$ and $O(tn)$, respectively.

2. Representation of t -ary trees

Wu et al. [18] defined a concise representation, called *right-distance sequences* (abbreviated as RD-sequences), to describe t -ary trees. Let T be a t -ary tree with n internal nodes numbered from 1 to n in preorder (i.e., visit the root and then recursively the subtrees of T from left to right). The *right-distance* of an internal node $i \in T$, denoted by d_i , is defined as follows:

$$d_i = \begin{cases} 0 & \text{if } i = 1 \text{ (i.e., the root of } T\text{);} \\ d_{p(i)} + t - k & \text{otherwise,} \end{cases}$$

where $p(i)$ stands for the parent of node i and k is the rank of node i among all sons of $p(i)$ (from left to right). Then, denote the sequence $rd(T) = (d_1, d_2, \dots, d_n)$ as the RD-sequence of T . A characterization has been pointed out in [18] that an integer sequence $(d_1, d_2, \dots, d_n)$ is the RD-sequence of a t -ary tree if and only if $d_1 = 0$ and $0 \leq d_i \leq d_{i-1} + t - 1$ for $i = 2, \dots, n$. For example, Figure 1 shows the 3-ary tree T with $rd(T) = (0, 2, 2, 4, 0, 0)$.

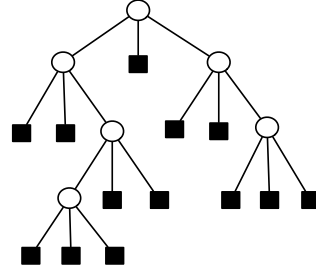


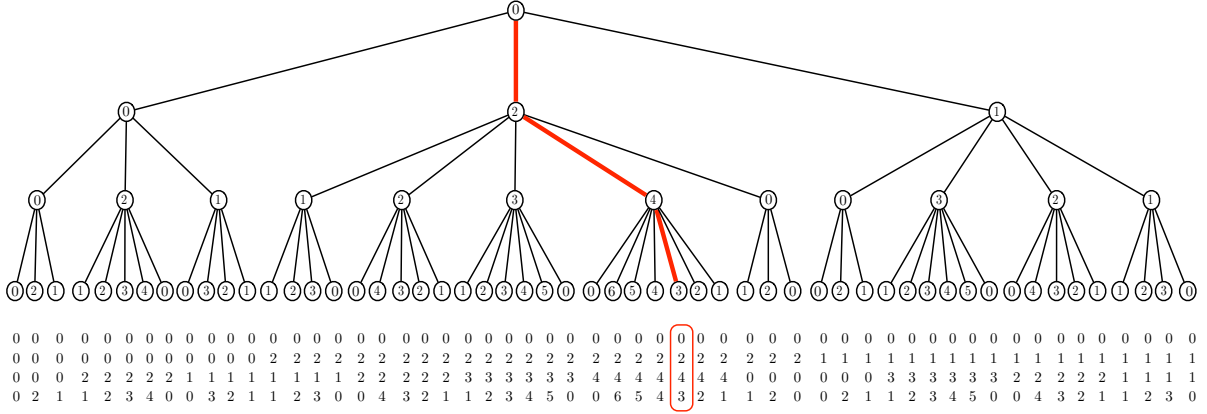
Figure 1: A 3-ary tree with 6 internal nodes.

Another way of presenting t -ary trees is the so-called *z-sequences* suggested by Zaks [20]. For each internal node $i \in T$, we denote z_i as the visited order of node i when both internal nodes and leaves of T are traversed in preorder. Then, the resulting sequence $z(T) = (z_1, z_2, \dots, z_n)$ is called the z -sequence of T . Note that every z -sequence satisfies $z_{i-1} < z_i \leq t(i-1) + 1$. For example, the z -sequence of the 3-ary tree T shown in Figure 1 is $z(T) = (1, 2, 5, 6, 13, 16)$. The following is the complementary relationship between RD-sequences and z -sequences.

Theorem 1 [18] *If a t -ary tree is encoded as $(d_1, d_2, \dots, d_n)$ for RD-sequence and $(z_1, z_2, \dots, z_n)$ for z -sequence, then $d_i + z_i = t(i-1) + 1$ for $i = 1, 2, \dots, n$.*

3. Loopless generation and Gray-code order

In this section, we give the concept about the loopless Gray-code generation algorithm for t -ary trees proposed in [17]. Let $\mathcal{T}_{t,n}$ denote the set of all t -ary trees with n internal nodes. Recall that $|\mathcal{T}_{t,n}| = \frac{1}{tn+1} \binom{tn+1}{n} = \frac{1}{(t-1)n+1} \binom{tn}{n}$. To describe all t -ary trees in a systematic way, a coding tree called *flip-flap tree* was introduced. A flip-flap tree, denoted by $\mathbb{T}_{t,n}$, is a rooted labeled tree with n levels constructed as follows:


 Figure 2: A flip-flap tree $\mathbb{T}_{3,4}$.

- (i) The first level contains the only one node (i.e., the root) with label 0;
- (ii) Every node with label d in the i th level, $i < n$, has $d + t$ sons labeled by $0, 1, \dots, d + t - 1$ in the next level, where the sons are arranged in either $1, 2, \dots, d + t - 1, 0$ (i.e., an up fragment) or $0, d + t - 1, \dots, 2, 1$ (i.e., a down fragment).
- (iii) If the sons of a node x with label d are arranged in an up fragment, then the sons of its adjacency siblings (i.e., nodes with labels $d - 1$ and $d + 1$ in the same level as x), if exist, are arranged in a down fragment, and vice versa.

Accordingly, nodes on the boundary in every fragment are labeled by either 0 or 1. And, the full labels along a path from the root to a leaf in $\mathbb{T}_{t,n}$ indicate the RD-sequence of a t -ary tree. Moreover, a property showed that any two adjacent RD-sequences in the flip-flap tree differ in exactly one digit. Therefore, this provides the base for designing a loopless Gray-code generation algorithm. For example, Figure 2 shows a flip-flap tree with respect to all 3-ary trees with 4 internal nodes, where the first fragment in each level is a down fragment. Thus, the first RD-sequence appears in $\mathbb{T}_{3,4}$ is $(0, 0, 0, 0)$. Also, a path with bold lines from the root to a leaf in $\mathbb{T}_{3,4}$ represents the 3-ary trees with RD-sequence $(0, 2, 4, 3)$.

A procedure called `NextRD()` is designed for generating the next RD-sequence in [17]. For t -ary trees, this procedure needs only $3n + O(1)$ space. Array $d[1..n]$ is used to store the current RD-sequence in the Gray-code order. Array $c[1..n]$ is for indicating the type of current fragment at level i for $1 \leq i \leq n$, where $c[i] = 1$ indicates an up fragment and $c[i] = -1$ indicates a down fragment. Array $F[1..n]$ is for keeping the track of positions to be processed. Initially, let $i = n$ and for each

$j = 1, 2, \dots, n$, set $d[j] = 0$, $c[j] = -1$ and $F[j] = j$. When each time `NextRD()` is invoked, $d[i]$ will be changed by increasing one or decreasing one which depends on the type of $c[i]$ (where arithmetic is taken modulo $d[i - 1] + t$). If the current fragment is faced on a boundary, it changes the type of $c[i]$, appoints the next changed position as $i = F[n]$, and performs the updates of $F[i]$, $F[i - 1]$ and $F[n]$; otherwise, it merely keeps the next changed position as $i = n$. The algorithm terminates when the condition $i = 1$ holds. The reader please refer to the details of the algorithm written by C++ in [17].

4. Ranking algorithm

We first observe that the largest label for nodes in the i th level of $\mathbb{T}_{t,n}$ is $(i - 1)(t - 1)$. Also, if two nodes have the same label in the same level of $\mathbb{T}_{t,n}$, then they have the same degree (i.e., the number of children). This further implies that the numbers of leaves in the subtrees rooted at each of the two nodes are the same. Let $A_{i,k}$ denote the number of leaves in the subtree rooted at a node with label k in the i th level of $\mathbb{T}_{t,n}$. Obviously, $A_{n,k} = 1$ for $0 \leq k \leq (n - 1)(t - 1)$ and $A_{n-1,k} = t + k$ for $0 \leq k \leq (n - 2)(t - 1)$. In general, we have

$$A_{i,k} = \sum_{j=0}^{t+k-1} A_{i+1,j} \quad (1)$$

where $1 \leq i \leq n - 1$ and $0 \leq k \leq (i - 1)(t - 1)$.

For the efficiency of computation, we define the following formula:

$$B_{i,k} = \sum_{j=0}^k A_{i,j} \quad (2)$$

where $1 \leq i \leq n$ and $0 \leq k \leq (i-1)(t-1)$. That is, the term $B_{i,k}$ indicates the total number of leaves for those subtrees rooted at nodes with labels from 0 to k in the i th level of $\mathbb{T}_{t,n}$. Hence, a table built from Eq. (2) is named as the *accumulation table* with respect to $\mathbb{T}_{t,n}$. Wu et al. [18] gave the following closed-form for calculating the accumulation table. For $1 \leq i \leq n$ and $0 \leq k \leq (i-1)(t-1)$,

$$B_{i,k} = \frac{k+1}{mt+k+1} \binom{mt+k+1}{m} \quad (3)$$

where $m = n-i+1$. For instance, the accumulation table with respect to $\mathbb{T}_{3,4}$ is shown below.

$B_{i,k}$		k						
		0	1	2	3	4	5	6
i	1	55						
	2	12	30	55				
	3	3	7	12	18	25		
	4	1	2	3	4	5	6	7

We are now in a position to determine the rank of a t -ary tree in the Gray-code order of [17]. For a given t -ary tree T associated with the RD-sequence $rd(T) = (d_1, d_2, \dots, d_n)$, let x_i denote the node with label d_i in the flip-flap tree $\mathbb{T}_{t,n}$ such that the full labels along the path $x_1, x_2, \dots, x_n$ represent the given RD-sequence. For each $i = 1, 2, \dots, n$, let $R(i)$ be the rank of x_i in the i th level of $\mathbb{T}_{t,n}$. Hereafter, the rank of a node in each level of $\mathbb{T}_{t,n}$ is ordered from left to right and a ranking always starts with 0. Note that $R(1) = 0$ and our ranking algorithm is to obtain $R(n)$.

Since each level of $\mathbb{T}_{t,n}$ begins with a down fragment and the two types of fragments appear alternately, it is easy to check the following property.

Proposition 1 For $1 \leq i < n$, if the rank of a node in the i th level of $\mathbb{T}_{t,n}$ is even (respectively, odd), then its sons are arranged in a down fragment (respectively, an up fragment).

For each $i = 1, \dots, n$, we first set $R(i) = 0$ and the computation of $R(i)$ requires $i-1$ updates by means of Eqs. (4) and (5). If $i \neq n$ and x_{i+1} is not the first node in a fragment, let y_{i+1} be the node with rank $R(i+1) - 1$ in the $(i+1)$ th level of $\mathbb{T}_{t,n}$. Then, for each $j = i+1, \dots, n$, we update $R(j)$ to represent the rank of the rightmost descendant of y_{i+1} in the j th level of $\mathbb{T}_{t,n}$ (including y_{i+1} itself if $j = i+1$). Let $\ell = n - j + i + 1$. By Proposition 1, there are two cases to compute $R(j)$. If $R(i) \equiv 0 \pmod{2}$, the sons of x_i are arranged in a down fragment. In this case, we have

$$R(j) = R(j) + (B_{\ell, d_i + (t-1)} - B_{\ell, d_i+1} + B_{\ell, 0}). \quad (4)$$

On the other hand, if $R(i) \equiv 1 \pmod{2}$, the sons of x_i are arranged in an up fragment. In this case, we have

$$R(j) = R(j) + (B_{\ell, d_i+1-1} - B_{\ell, 0}). \quad (5)$$

The last term of the right hand side in each of Eqs. (4) and (5) is called the *incremental change* of the update. The meanings of incremental changes are illustrated in Figure 3 and Figure 4, respectively.

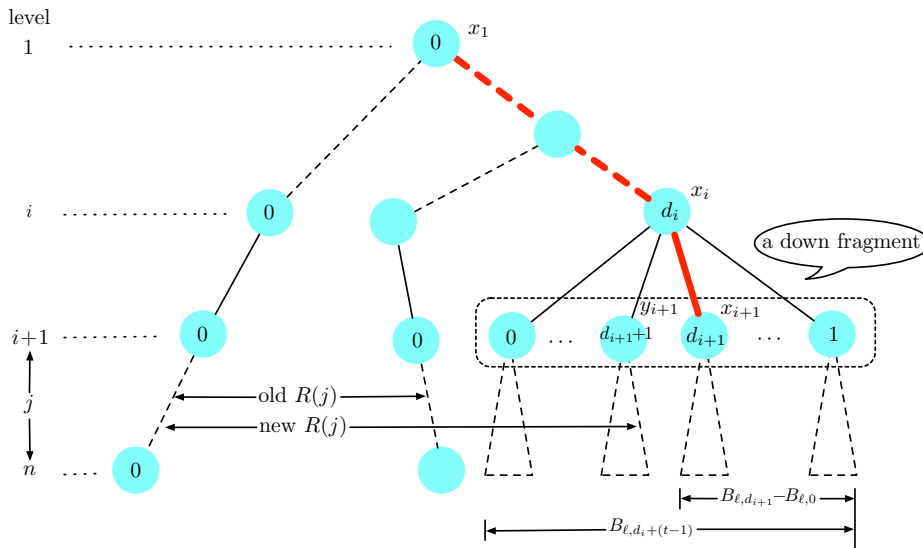


Figure 3: Illustration of Eq. (4).

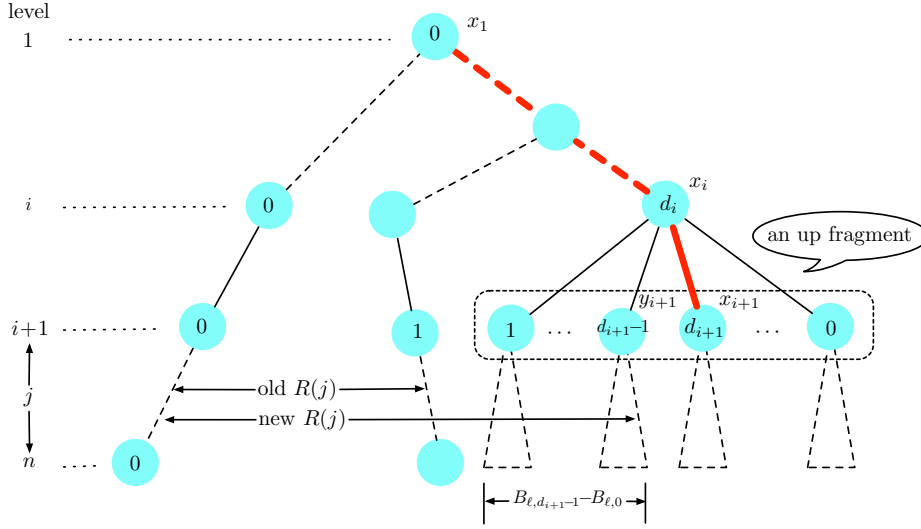


Figure 4: Illustration of Eq. (5).

According to Eqs. (4) and (5), we design the following algorithm.

Function Ranking($d_1, d_2, \dots, d_n$)

```

begin
  for  $i = 1$  to  $n$  do  $R(i) = 0$ ;
  for  $i = 1$  to  $n - 1$  do
    if  $R(i) \equiv 0 \pmod{2}$  then
      // a down fragment
      if  $d_{i+1} \neq 0$  then
        for  $j = i + 1$  to  $n$  do
           $\ell = n - j + i + 1$ ;
           $R(j) = R(j) + (B_{\ell, d_i + (t-1)} - B_{\ell, d_{i+1}} + B_{\ell, 0})$ ;
      else // an up fragment
        if  $d_{i+1} \neq 1$  then
          for  $j = i + 1$  to  $n$  do
             $\ell = n - j + i + 1$ ;
             $R(j) = R(j) + (B_{\ell, d_{i+1}-1} - B_{\ell, 0})$ ;
  return  $R(n)$ ;
    
```

Example 1. We consider a 3-ary tree T with $rd(T) = (0, 2, 4, 3)$ and perform Ranking($0, 2, 4, 3$). Initially, $R(1) = R(2) = R(3) = R(4) = 0$.

When $i = 1$, since $R(1) = 0$ is even, the sons of x_1 are arranged in a down fragment. Thus, we have

$$R(2) = R(2) + B_{4,2} - B_{4,2} + B_{4,0} = 0 + 3 - 3 + 1 = 1,$$

$$R(3) = R(3) + B_{3,2} - B_{3,2} + B_{3,0} = 0 + 12 - 12 + 3 = 3,$$

$$R(4) = R(4) + B_{2,2} - B_{2,2} + B_{2,0} = 0 + 55 - 55 + 12 = 12.$$

When $i = 2$, since $R(2) = 1$ is odd, the sons of x_2 are arranged in an up fragment. Then, we have the following updates:

$$R(3) = R(3) + B_{4,3} - B_{4,0} = 3 + 4 - 1 = 6,$$

$$R(4) = R(4) + B_{3,3} - B_{3,0} = 12 + 18 - 3 = 27.$$

When $i = 3$, since $R(3) = 6$ is even, the sons of x_3 are arranged in a down fragment. We obtain

$$R(4) = R(4) + B_{4,6} - B_{4,3} + B_{4,0} = 27 + 7 - 4 + 1 = 31.$$

Finally, the algorithm outputs $R(4) = 31$. $\square$

Obviously, building the accumulation table using Eq. (3) requires $O(tn^2)$ time and space. In what follows, we will show that every element of the accumulation table in the ranking algorithm can be accessed in a constant time provided we make a preprocessing in advance. The basic ideal is that all accessed elements in the accumulation table always occur in a set of certain columns when every time we update $R(j)$ in the ranking algorithm, e.g. three columns for a down fragment and two columns for an up fragment. Thus, we can easily calculate elements of a column in a sequence. In particular, using Eq. (3), we can obtain the following formula:

$$B_{i-1,k} = \frac{B_{i,k}}{m+1} \cdot \frac{\prod_{j=0}^{t-1} mt + k + 1 + j}{\prod_{j=1}^{t-1} m(t-1) + k + 1 + j} \quad (6)$$

where $m = n - i + 1$.

Eq. (6) states the relation between two elements

$B_{i-1,k}$ and $B_{i,k}$ in the same column of the table. In addition, we observe that the numerator (respectively, the denominator) of a fractional number in the right hand side includes the product of t (respectively, $t-1$) consecutive integers. This inspires us to build a table of size $(n-1)(t-1) \times 3$ to store the terms of all products. For example, for $n = 4$ and $t = 3$, the desired table is as follows.

x	$f(x) = x(x+1)(x+2)$	$g(x) = x(x+1)$
4	120	20
5	210	30
6	336	42
7	504	56
8	720	72
9	990	90

Here, the first term $x = 4$ is due to the initial conditions $m = 1$ and $k = 0$. In this case, we have $\prod_{j=0}^{t-1} mt + k + 1 + j = 4 \cdot 5 \cdot 6 = 120$ and $\prod_{j=1}^{t-1} m(t-1) + k + 1 + j = 4 \cdot 5 = 20$. Clearly, $f(x+1) = f(x) \cdot \frac{(x+t)}{x}$ and $g(x+1) = g(x) \cdot \frac{(x+t-1)}{x}$. Thus, the table can be constructed in $O(tn)$ time.

Since every element in the last row of the accumulation table is easy to obtain (i.e., $B_{n,k} = k+1$), we can compute all elements in the same column by using the preprocessing table. For example, we have known $B_{4,2} = 3$. Since $i = 4$ (i.e., $m = 1$) and $k = 2$ in this stage, both the terms $mt + k + 1 + j$ where $j = 0$ and $m(t-1) + k + 1 + j$ where $j = 1$ are equal to 6. Thus, using the terms as indices to search for the preprocessing table, we can compute $B_{3,2}$ as follows:

$$B_{3,2} = \frac{B_{4,2}}{m+1} \cdot \frac{f(6)}{g(6)} = \frac{3}{2} \cdot \frac{336}{42} = 12.$$

Next, since the current stage has changed i to 3 (i.e., $m = 2$), the term $mt + k + 1 + j$ where $j = 0$ (respectively, $m(t-1) + k + 1 + j$ where $j = 1$) can be obtained from its previous value by adding an extra offset t (respectively, $t-1$). Thus, the two indices are $6 + 3 = 9$ and $6 + 2 = 8$, respectively. We now compute $B_{2,2}$ as follows:

$$B_{2,2} = \frac{B_{3,2}}{m+1} \cdot \frac{f(9)}{g(8)} = \frac{12}{3} \cdot \frac{990}{72} = 55.$$

We repeat such a process until all required elements in the same column are calculated. Since building the preprocessing table needs $O(tn)$ time and space and the ranking algorithm can be run in $O(n^2)$ time, we conclude the following.

Theorem 2 *Determining the rank of a t -ary tree with n internal nodes in a Gray-code order can be done in $O(\max\{n^2, tn\})$ time and $O(tn)$ space.*

5. Conclusion and future work

In this paper, we present a ranking algorithm of t -ary trees with n internal nodes encoded by RD-sequences in a Gray-code order. The algorithms can be run in $O(\max\{n^2, tn\})$ time and $O(tn)$ space, respectively. As we have mentioned earlier, Roelants van Baronaigien [10] and Xiang et al. [19] independently introduced a loopless Gray-code generation of t -ary trees encoded by z-sequences. Based on such a Gray-code order, Ahmadi-Adl et al. [2] recently proposed ranking and unranking algorithms that each can be run in $O(tn^2)$ time. Although it has been pointed out in [17] that there are at least 2^{n-1} different ways to generate Gray-codes of t -ary trees with n internal nodes, we found that z-sequences generated in [10, 19] and RD-sequences generated in [17] indeed realize the complementary relationship (as stated in Theorem 1) for each generation. As a result, with a slight modification, our algorithm could make an improvement for ranking t -ary trees encoded by z-sequences in a Gray-code order.

As a future work, we will design an efficient unranking algorithm. Also, another interesting and challenging problem is how to extend our proposed algorithm for dealing with non-regular trees encoded by RD-sequences in a Gray-code order and preserves the lower complexities of time and space. To the best of our knowledge, so far no algorithms exist to solve such ranking and unranking problems.

References

- [1] H. Ahrabian and A. Nowzari-Dalini, Generation of t -ary trees with ballot-sequences, *International Journal of Computer Mathematics* 80 (2003) 1243–1249.
- [2] A. Ahmadi-Adl, A. Nowzari-Dalini, and H. Ahrabian, Ranking and unranking algorithms for loopless generation of t -ary trees, *Logic Journal of IGPL*, 19 (2011) 33–43.
- [3] M.C. Er, A note on generating well-formed parenthesis strings lexicographically, *The Computer Journal* 26 (1983) 205–207.
- [4] M.C. Er, Lexicographic listing and ranking t -ary trees, *The Computer Journal* 30 (1987) 569–572.
- [5] J.F. Korsh, Loopless generation of k -ary tree sequences, *Information Processing Letters* 52 (1994) 243–247.

- [6] J.F. Korsh and P. LaFollette, Loopless generation of Gray codes for k -ary trees, *Information Processing Letters* 70 (1999) 7–11.
- [7] J.F. Korsh and P. LaFollette, A loopless Gray code for rooted trees, *ACM Transactions on Algorithms* 2 (2006) 135–152.
- [8] J.F. Korsh and S. Lipschutz, Shift and loopless generation of k -ary trees, *Information Processing Letters* 65 (1998) 235–240.
- [9] A. Proskurowski and F. Ruskey, Binary tree Gray codes, *Journal of Algorithms* 6 (1985) 225–238.
- [10] D. Roelants van Baronaigien, A loopless Gray-code algorithm for listing k -ary trees, *Journal of Algorithms* 35 (2000) 100–107.
- [11] D. Roelants van Baronaigien and F. Ruskey, Generating t -ary trees in A-order, *Information Processing Letters* 27 (1988) 205–213.
- [12] F. Ruskey, Generating t -ary trees lexicographically, *SIAM Journal on Computing* 7 (1978) 424–439.
- [13] C.D. Savage, A survey of combinatorial Gray codes, *SIAM Review* 39 (1997) 605–629.
- [14] A.E. Trojanowaki, Ranking and listing algorithms for k -ary trees, *SIAM Journal on Computing* 7 (1978) 492–509.
- [15] V. Vajnovszki, Generating a Gray code for P-sequences, *Journal of Mathematical Modelling and Algorithms* 1 (2002) 31–41.
- [16] R.-Y. Wu, J.-M. Chang, and C.-H. Chang, Ranking and unranking of non-regular trees with a prescribed branching sequence, *Mathematical and Computer Modelling* 53 (2011) 1331–1335.
- [17] R.-Y. Wu, J.-M. Chang, and Y.-L. Wang, Loopless Generation of non-regular trees with a prescribed branching sequence, *The Computer Journal* 53 (2010) 661–666.
- [18] R.-Y. Wu, J.-M. Chang, and Y.-L. Wang, Ranking and unranking of t -ary trees using RD-sequences, *IEICE Transactions on Information and Systems*, E94-D (2011) 226–232.
- [19] L. Xiang, K. Ushijima, and C. Tang, Efficient loopless generation of Gray codes for k -ary trees, *Information Processing Letters* 76 (2000) 169–174.
- [20] S. Zaks, Lexicographic generation of ordered trees, *Theoretical Computer Science* 10 (1980) 63–82.
- [21] S. Zaks, Generation and ranking of k -ary trees, *Information Processing Letters* 14 (1982) 44–48.